



**UNIVERSIDADE FEDERAL DO TOCANTINS  
CÂMPUS UNIVERSITÁRIO DE PALMAS  
CURSO DE CIÊNCIA DA COMPUTAÇÃO**

**HENRIQUE NORONHA FERNANDES**

**ANÁLISE COMPARATIVA DE GERENCIAMENTO DE RECURSOS DO  
SO: ABLETON LIVE VS. FL STUDIO**

**PALMAS (TO)**

**2026**

**Henrique Noronha Fernandes**

**Análise Comparativa de Gerenciamento de Recursos do SO: Ableton Live vs.  
FL Studio**

Relatório de Sistemas Operacionais  
apresentado à Universidade Federal do  
Tocantins para obtenção do título de Bacharel  
em Ciência da Computação. Orientador:

[Titulação] [Nome do Professor de SO]  
[Sobrenome]

PALMAS (TO)

2026

## SUMÁRIO

|            |                                                                    |           |
|------------|--------------------------------------------------------------------|-----------|
| <b>1</b>   | <b>INTRODUÇÃO . . . . .</b>                                        | <b>0</b>  |
| <b>2</b>   | <b>REFERENCIAL TEÓRICO . . . . .</b>                               | <b>1</b>  |
| <b>2.1</b> | <b>Gerenciamento de Processos e <i>Threads</i> . . . . .</b>       | <b>1</b>  |
| 2.1.1      | Escalonador do Windows NT . . . . .                                | 1         |
| 2.1.2      | Tempo Real Crítico vs. Não Crítico . . . . .                       | 1         |
| <b>2.2</b> | <b>Interrupções e Latência de I/O em Tempo Real . . . . .</b>      | <b>2</b>  |
| 2.2.1      | <i>Callback</i> de Áudio e Tamanho de <i>Buffer</i> . . . . .      | 2         |
| 2.2.2      | ASIO vs. WDM/DirectSound . . . . .                                 | 2         |
| 2.2.3      | Algoritmos Formais de Escalonamento em Tempo Real . . . . .        | 2         |
| <b>2.3</b> | <b>Gerenciamento de Memória e Paginação . . . . .</b>              | <b>3</b>  |
| 2.3.1      | <i>Eager</i> vs. <i>Lazy Allocation</i> . . . . .                  | 3         |
| 2.3.2      | <i>Private Bytes</i> . . . . .                                     | 3         |
| <b>3</b>   | <b>METODOLOGIA . . . . .</b>                                       | <b>4</b>  |
| <b>4</b>   | <b>RESULTADOS . . . . .</b>                                        | <b>5</b>  |
| 4.1        | Testes 1 e 2 — Consumo em Repouso e Carga <i>I/O-bound</i> . . .   | 5         |
| 4.2        | Teste 3 — Escalonamento de <i>Threads</i> . . . . .                | 8         |
| 4.3        | Teste 4 — Latência com Driver Nativo . . . . .                     | 9         |
| 4.4        | Teste 5 — Estresse CPU-bound e Ponto de Ruptura . . . . .          | 9         |
| <b>5</b>   | <b>DISCUSSÃO . . . . .</b>                                         | <b>11</b> |
| 5.1        | Análise Comparativa — Teste 1 (Repouso) . . . . .                  | 11        |
| 5.2        | Análise Comparativa — Teste 2 (20 Trilhas AIFF) . . . . .          | 11        |
| 5.3        | Análise Comparativa — Teste 3 (Escalonamento de <i>Threads</i> ) . | 11        |
| 5.4        | Análise Comparativa — Teste 4 (Latência) . . . . .                 | 11        |

|     |                                                              |    |
|-----|--------------------------------------------------------------|----|
| 5.5 | Análise Comparativa — Teste 5 (Estresse CPU-bound) . . . . . | 12 |
| 6   | CONCLUSÃO . . . . .                                          | 13 |
|     | REFERÊNCIAS . . . . .                                        | 14 |

## 1 INTRODUÇÃO

Historicamente, os Sistemas Operacionais (SO) evoluíram de simples monitores de execução sequencial para ecossistemas complexos de gerenciamento de hardware e software. Enquanto as primeiras gerações de sistemas focavam estritamente na abstração de comandos básicos e organização de arquivos, os sistemas contemporâneos precisam orquestrar uma arquitetura de múltiplos núcleos (*multicore*), memória virtual e interrupções em milissegundos. No cenário contemporâneo, os computadores pessoais atuam como Estações de Trabalho de Áudio Digital (DAWs — *Digital Audio Workstations*), executando softwares complexos que exigem do SO uma orquestração extremamente eficiente dos componentes de hardware. Nesse contexto, a performance de uma DAW não depende apenas da força bruta do processador, mas de como o sistema operacional gerencia as *threads* de execução, aloca a memória e lida com operações de entrada e saída (I/O) em alta velocidade.

O grande desafio imposto por essas aplicações é a necessidade de operar sob restrições de tempo real. Conforme a classificação de Silberschatz, Galvin e Gagne (2015), as DAWs enquadram-se como sistemas de tempo real não crítico (*soft real-time*). Nesses sistemas, embora o atraso no atendimento de um processo não cause uma falha fatal no computador, ele resulta em uma severa degradação da qualidade do serviço — nesse caso, manifestando-se como falhas e artefatos sonoros indesejados. Para evitar essa degradação, o sistema operacional precisa garantir que as *threads* de áudio tenham preferência sobre processos comuns, minimizando a latência de despacho e a latência de interrupção.

Diante dessa problemática, o presente trabalho realiza uma análise comparativa do consumo e gerenciamento de recursos do sistema operacional entre duas das principais DAWs do mercado: Ableton Live e FL Studio. Busca-se mapear como cada arquitetura de software impacta o consumo de CPU, o comportamento *multithread*, o uso de memória RAM e a latência de I/O sob as mesmas condições de estresse, demonstrando na prática os impactos das decisões de escalonamento e arquitetura do SO em ambientes de alta exigência em tempo real.

## 2 REFERENCIAL TEÓRICO

### 2.1 Gerenciamento de Processos e *Threads*

Um processo é um programa que está sendo executado pelo sistema operacional, com espaço de endereçamento próprio e os recursos atribuídos pelo SO (SILBERSCHATZ; GALVIN; GAGNE, 2015). Dentro de um mesmo processo podem existir várias *threads*: unidades menores de execução que compartilham o espaço de endereçamento do processo, mas têm pilha e registradores próprios (TANENBAUM; BOS, 2016). Como elas compartilham memória, a troca de contexto entre *threads* é bem mais barata do que entre processos. Por causa disso, motores de áudio costumam usar *multithreading* para dividir o processamento de blocos de amostras entre os núcleos do hardware.

#### 2.1.1 Escalonador do Windows NT

O escalonador do Windows NT é preemptivo e baseado em prioridades, com 32 níveis numerados de 0 a 31. Os níveis 1 a 15 formam a faixa de usuário e os níveis 16 a 31 formam a faixa de tempo real (STALLINGS, 2017). Os valores observados nos testes deste trabalho ficam todos na faixa de usuário: *Normal* (8), *Above Normal* (10) e *High* (12 e 13).

Cada *thread* tem uma prioridade base, derivada da classe do processo. Em tempo de execução, o sistema pode aplicar um *priority boost* temporário para evitar *starvation* ou premiar *threads* que acabaram de acordar de uma operação de I/O (STALLINGS, 2017). Esse *boost* vai caindo até a *thread* voltar à prioridade base, e isso explica por que o Process Explorer mostra valores diferentes para prioridade base e prioridade dinâmica.

#### 2.1.2 Tempo Real Crítico vs. Não Crítico

Silberschatz, Galvin e Gagne (2015) dividem os sistemas de tempo real em dois tipos. No tempo real crítico (*hard real-time*), perder um prazo é uma falha grave, inaceitável em aplicações como aviãoica ou equipamento médico. No tempo real não crítico (*soft real-time*), perder um prazo eventualmente atrapalha a qualidade do serviço, mas não compromete o sistema como um todo.

As DAWs ficam nesse segundo grupo. Quando um *buffer* de áudio não é preenchido a tempo, o usuário ouve um *dropout*, mas a máquina continua estável. Para garantir prioridade sem precisar de garantia formal, basta colocar as *threads* de áudio em prioridade maior que a dos processos comuns. Assim, o escalonador as despacha primeiro quando há disputa por CPU.

## 2.2 Interrupções e Latência de I/O em Tempo Real

Uma interrupção de hardware é um sinal assíncrono que um periférico envia ao processador para avisar que precisa de atenção imediata (SILBERSCHATZ; GALVIN; GAGNE, 2015). Quando o sinal chega, o processador para a *thread* que está executando, salva seu estado e transfere o controle para a Rotina de Serviço de Interrupção (ISR). O intervalo entre o sinal e o início da ISR é chamado de latência de interrupção. Somando esse intervalo ao tempo de troca de contexto até a *thread* destino, temos a latência de despacho (STALLINGS, 2017). No caso do áudio, se essa latência ultrapassar o período do *buffer*, aparece silêncio ou artefato sonoro na saída.

### 2.2.1 *Callback* de Áudio e Tamanho de *Buffer*

Drivers de áudio modernos funcionam no modelo *pull callback*: o hardware solicita periodicamente um bloco de  $N$  amostras, e o motor de áudio precisa entregá-lo antes do próximo bloco ser pedido (FONS, 2010). A relação entre tamanho do *buffer* e latência é dada por:

$$L = \frac{N}{f_s} \quad (1)$$

onde  $L$  é a latência em segundos,  $N$  o tamanho do bloco em amostras e  $f_s$  a taxa de amostragem. Por exemplo, com  $f_s = 44.100$  Hz e  $N = 256$ , a latência fica em torno de 5,8 ms. Reduzir  $N$  diminui a latência percebida, mas aumenta a frequência das interrupções e, com isso, a carga sobre o escalonador (KIM; HAHN, 2003).

### 2.2.2 ASIO vs. WDM/DirectSound

O Windows oferece dois caminhos principais para áudio. No caminho WDM/DirectSound, o sinal passa pelo *Kernel Audio Mixer* (KMixer), que junta os fluxos de vários aplicativos antes de enviar para o hardware (Microsoft Corporation, 2023a). Como o KMixer precisa ser acordado a cada ciclo de *buffer*, isso acaba adicionando latência e gerando interrupções extras.

Já o protocolo ASIO (*Audio Stream Input/Output*) dá ao driver do fabricante acesso direto ao hardware, pulando o KMixer (Microsoft Corporation, 2023a). O resultado é menos latência e menos pressão sobre o escalonador.

### 2.2.3 Algoritmos Formais de Escalonamento em Tempo Real

Vale lembrar que existe uma linha de pesquisa formal sobre escalonamento de prazos em tempo real. Liu e Layland (1973) formalizaram o problema com o algoritmo *Rate Monotonic*, e Buttazzo (2011) mostraram que garantir prazos formalmente exige análise de pior caso (WCET). As DAWs, por serem *soft real-time*, não usam esse rigor.

Elas apenas aumentam a prioridade das *threads* de áudio no escalonador comum do SO, abrindo mão da garantia formal em troca de simplicidade de implementação.

## 2.3 Gerenciamento de Memória e Paginação

Cada processo recebe do SO um espaço de endereçamento virtual contíguo e isolado, independente da memória física disponível (SILBERSCHATZ; GALVIN; GAGNE, 2015). A tradução entre endereço virtual e físico é feita pela MMU, usando as tabelas de páginas mantidas pelo *kernel*. Na paginação por demanda, uma página só é mapeada para um quadro físico (*frame*) quando é acessada pela primeira vez. Esse evento é chamado de falta de página (*page fault*) (TANENBAUM; BOS, 2016). Para resolver uma falta de página, o *kernel* suspende a *thread*, localiza a página (ou lê do disco se necessário) e retoma a execução. O tempo total dessa operação é não determinístico e pode variar de microssegundos a milissegundos, dependendo de onde a página estava.

Para áudio em tempo real, uma falta de página no meio do processamento de um bloco é especialmente cara: a *thread* fica bloqueada esperando o *kernel* resolver, enquanto o prazo do *buffer* continua correndo (KIM; HAHN, 2003). Sistemas críticos resolvem esse problema fixando páginas na RAM com `mlock()` no Linux ou `VirtualLock()` no Windows. As DAWs comerciais costumam adotar uma estratégia diferente: alocam boa parte da memória logo na inicialização, o que tem efeito parecido na prática.

### 2.3.1 *Eager vs. Lazy Allocation*

Tanenbaum e Bos (2016) contrastam dois modelos de alocação. Na *eager allocation*, o processo reserva e inicializa toda a memória prevista logo no início. Isso elimina faltas de página no caminho crítico, mas mantém uma pegada de memória grande mesmo em repouso. Já na *lazy allocation*, os recursos são alocados só conforme vão sendo demandados; a pegada em repouso fica mínima, mas pode haver latência não determinística no momento da primeira alocação. Como será discutido no Capítulo 5, esse contraste aparece de forma bem clara nos resultados deste trabalho.

### 2.3.2 *Private Bytes*

O contador *Private Bytes* do Windows mede a memória virtual comprometida exclusivamente pelo processo, sem contar DLLs e outras regiões compartilhadas (STALLINGS, 2018). Essa é a métrica que melhor reflete o custo real de memória da aplicação, já que mostra o que precisaria ser paginado para disco caso o sistema ficasse sem RAM física. No caso das DAWs, o crescimento desse contador ao carregar trilhas vem principalmente dos *buffers* de pré-leitura e das estruturas de decodificação que precisam ficar em memória para garantir reprodução de baixa latência.

### 3 METODOLOGIA

O estudo adota abordagem quantitativa e comparativa para analisar o gerenciamento de recursos de *Digital Audio Workstations* (DAWs) sob a ótica de Sistemas Operacionais, focando na interação entre o *kernel* do Windows 11 (build 26200, x64) e os motores de áudio do **Ableton Live 12** e do **FL Studio 24**. Os experimentos foram conduzidos em um notebook ASUS Vivobook K3605ZF equipado com processador Intel Core i5-12500H (12 núcleos, 16 *threads*, base 3,1 GHz), 16 GB de RAM DDR4 e GPU NVIDIA (driver `nvlddmkm.sys` v581.29). Para preservar as escolhas arquiteturais nativas de cada fabricante, o FL Studio operou com o driver *FL Studio ASIO* e o Ableton Live com *MME/DirectX*; o *Fruity Limiter* do canal Master do FL Studio foi removido antes dos testes para garantir paridade de cadeia de sinal. O Windows Defender permaneceu ativo em todas as sessões, em condições idênticas para ambas as DAWs.

A coleta de dados foi realizada com três ferramentas complementares. O **Performance Monitor** (PerfMon) capturou contadores do *kernel* a 1 Hz — % Tempo de Processador, Private Bytes e Contagem de Threads por processo — exportados via `relog` para formato `.csv` (Microsoft Corporation, 2023b). O **Process Explorer** (Sysinternals) forneceu inspeção das *threads* individuais, suas prioridades de escalonamento e consumo de CPU por unidade de execução. O **LatencyMon v7.31** (BERNSEE, 2023) mediu latência *interrupt-to-process*, tempos de rotinas DPC e ISR e contagem de *hard pagefaults* em nível de sistema.

O protocolo experimental compreendeu cinco testes sequenciais. O **Teste 1** (repouso) coletou dados com a DAW aberta e sem projeto carregado por 2,5 minutos. O **Teste 2** (carga *I/O-bound*) adicionou a reprodução simultânea em *loop* de 20 trilhas AIFF (PCM não comprimido) por 3 minutos. O **Teste 3** (escalonamento) inspecionou via Process Explorer a composição, prioridade e distribuição de *threads* com o projeto ativo. O **Teste 4** (latência) executou o LatencyMon por 2 min 50 s sobre o mesmo projeto, registrando DPCs, ISRs e *pagefaults*. Por fim, o **Teste 5** (estresse CPU-bound) introduziu cinco instâncias do reverb nativo de cada DAW no canal Master e reduziu progressivamente o *buffer size* de 4096 até 128 *samples*, identificando o ponto de ruptura (*buffer underrun*) pela ocorrência de *dropouts* audíveis.

## 4 RESULTADOS

### 4.1 Testes 1 e 2 — Consumo em Repouso e Carga *I/O-bound*

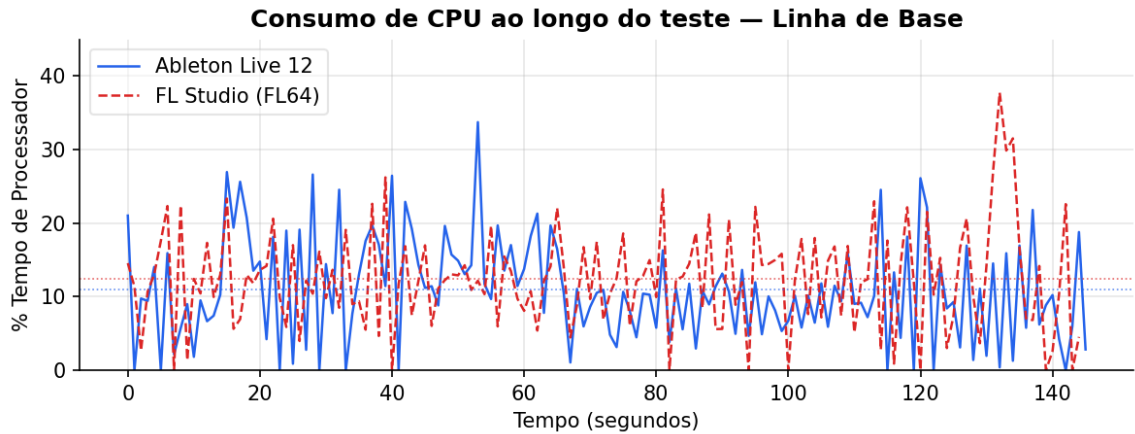
Os Testes 1 e 2 avaliaram o comportamento de CPU, memória e *threads* em dois cenários consecutivos: DAW em repouso (sem projeto) e com reprodução simultânea de 20 trilhas AIFF em *loop*. Os dados, amostrados a 1 Hz via PerfMon durante 2,5 e 3 minutos respectivamente, estão consolidados nas Tabelas 1 e ??.

**Tabela 1 – Resumo estatístico — Teste 1 (repouso) e Teste 2 (20 trilhas AIFF)**

| Teste           | Métrica            | DAW              | Mín.  | Média  | Máx.   |
|-----------------|--------------------|------------------|-------|--------|--------|
| T1 — Repouso    | % CPU              | Ableton Live 12  | 0,00% | 10,95% | 33,73% |
|                 |                    | FL Studio (FL64) | 0,00% | 12,38% | 37,72% |
|                 | Private Bytes (MB) | Ableton Live 12  | 338,6 | 536,8  | 538,3  |
|                 |                    | FL Studio (FL64) | 95,6  | 95,8   | 95,8   |
|                 | Threads ativas     | Ableton Live 12  | 8     | 54,8   | 56     |
|                 |                    | FL Studio (FL64) | 44    | 44,0   | 45     |
| T2 — 20 trilhas | % CPU              | Ableton Live 12  | 0,00% | 3,74%  | 20,11% |
|                 |                    | FL Studio (FL64) | 0,00% | 4,80%  | 31,16% |
|                 | Private Bytes (MB) | Ableton Live 12  | 601,6 | 601,7  | 604,5  |
|                 |                    | FL Studio (FL64) | 137,6 | 147,6  | 147,6  |
|                 | Threads ativas     | Ableton Live 12  | 54    | 55,3   | 56     |
|                 |                    | FL Studio (FL64) | 26    | 27,0   | 28     |

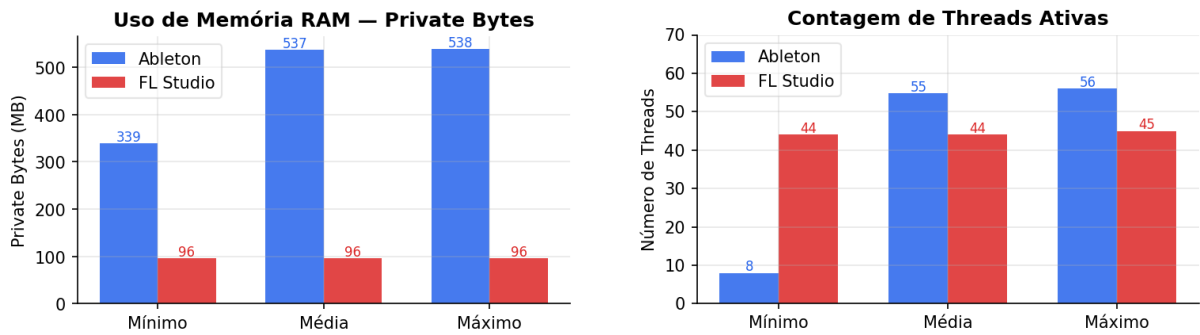
Fonte: dados coletados pelos autores via PerfMon, 26 maio 2026.

Em repouso, o FL Studio consumiu CPU média ligeiramente superior (12,38% vs. 10,95%), enquanto o Ableton Live alocou 5,6× mais memória (536,8 MB vs. 95,8 MB) e manteve o dobro de *threads* ativas (54,8 vs. 44,0). Com 20 trilhas em reprodução, ambas as DAWs reduziram o consumo médio de CPU — Ableton de 10,95% para 3,74% e FL Studio de 12,38% para 4,80% — ao passo que a memória cresceu moderadamente (+64,9 MB no Ableton, +51,8 MB no FL Studio). O FL Studio reduziu sua contagem de *threads* de 44 para 27 ao carregar o projeto (−38%), enquanto o Ableton permaneceu estável (54,8 → 55,3). As Figuras 1–5 ilustram esses comportamentos.



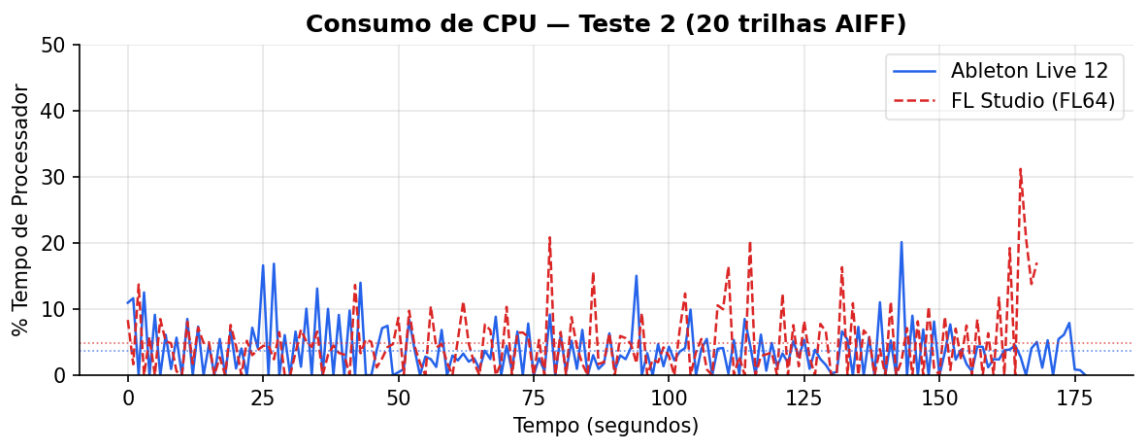
**Figura 1 – CPU em repouso ao longo do Teste 1.**

Fonte: dados coletados pelos autores via PerfMon, 26 maio 2026.



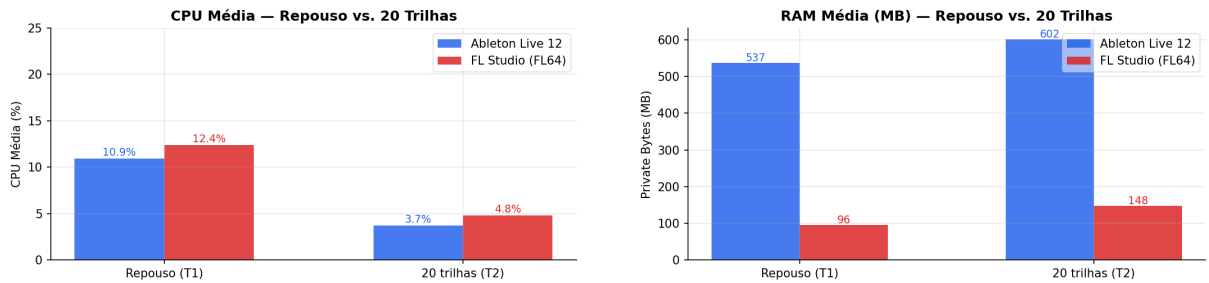
**Figura 2 – RAM (*Private Bytes*) e *threads* em repouso — Teste 1.**

Fonte: dados coletados pelos autores via PerfMon, 26 maio 2026.



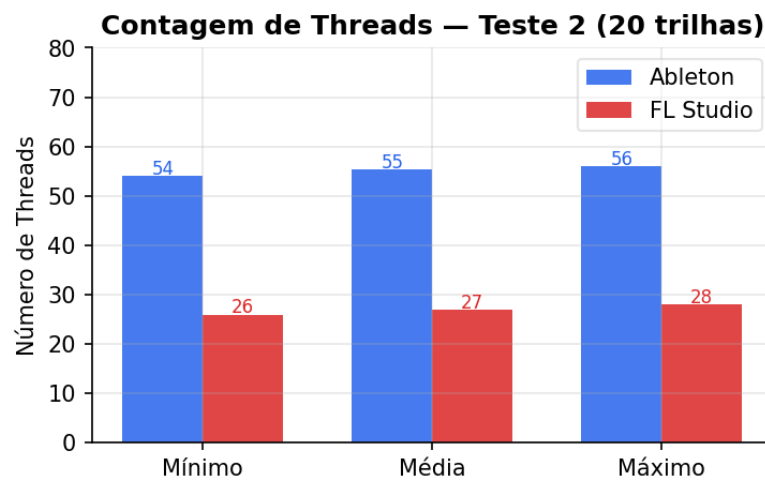
**Figura 3 – CPU durante reprodução de 20 trilhas AIFF — Teste 2.**

Fonte: dados coletados pelos autores via PerfMon, 26 maio 2026.



**Figura 4 – CPU e RAM médias comparativas: repouso (T1) vs. 20 trilhas (T2).**

Fonte: dados coletados pelos autores via PerfMon, 26 maio 2026.



**Figura 5 – Contagem de *threads* com 20 trilhas ativas — Teste 2.**

Fonte: dados coletados pelos autores via PerfMon, 26 maio 2026.

4.2 Teste 3 — Escalonamento de *Threads*

A inspeção via Process Explorer com o projeto em reprodução revelou diferenças relevantes na composição e prioridade de *threads* (Tabela 2). O FL Studio opera com prioridade base 10 (*Above Normal*) e dinâmica 12 (*High*), contra base 8 (*Normal*) e dinâmica 10 do Ableton Live. Nenhuma das DAWs atingiu o nível *Time Critical* (15). Em relação à composição, 14 das 29 *threads* do FL Studio pertencem ao módulo QuickFontCache (renderização de UI), com apenas 5 associadas ao FLEngine\_x64 (motor de áudio); no Ableton Live,  $\approx 30$  das 55 *threads* provêm do ucrtbase.dll (*thread pool* do C runtime). As Figuras 6 e 7 detalham esses dados.

Tabela 2 – Escalonamento de *threads* — Teste 3 (Process Explorer)

| Parâmetro                          | FL Studio (FL64)  | Ableton Live 12   |
|------------------------------------|-------------------|-------------------|
| Total de <i>threads</i>            | 29                | 55                |
| Prioridade base (thread principal) | 10 (Above Normal) | 8 (Normal)        |
| Prioridade dinâmica                | 12 (High)         | 10 (Above Normal) |
| CPU da thread principal            | 0,93%             | 0,75%             |
| Thread <i>Time Critical</i> (15)?  | Não               | Não               |
| Módulo dominante                   | QuickFontCache    | ucrtbase          |
| Thread de driver de áudio          | ilwasapi2asio_x64 | DSOUND.dll        |

Fonte: dados coletados pelos autores via Process Explorer, 26 maio 2026.

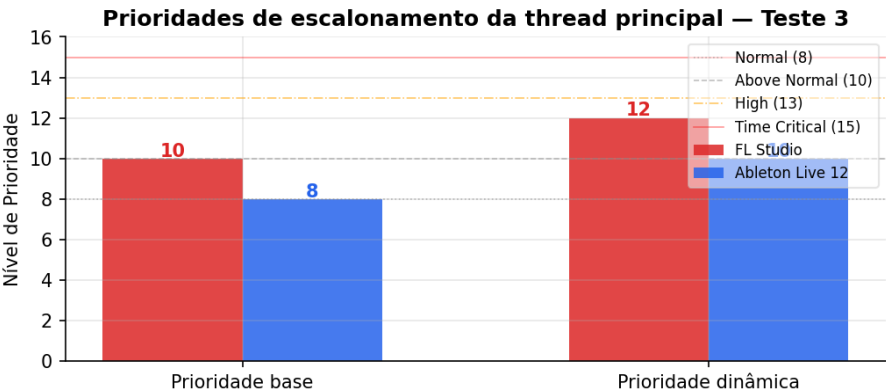
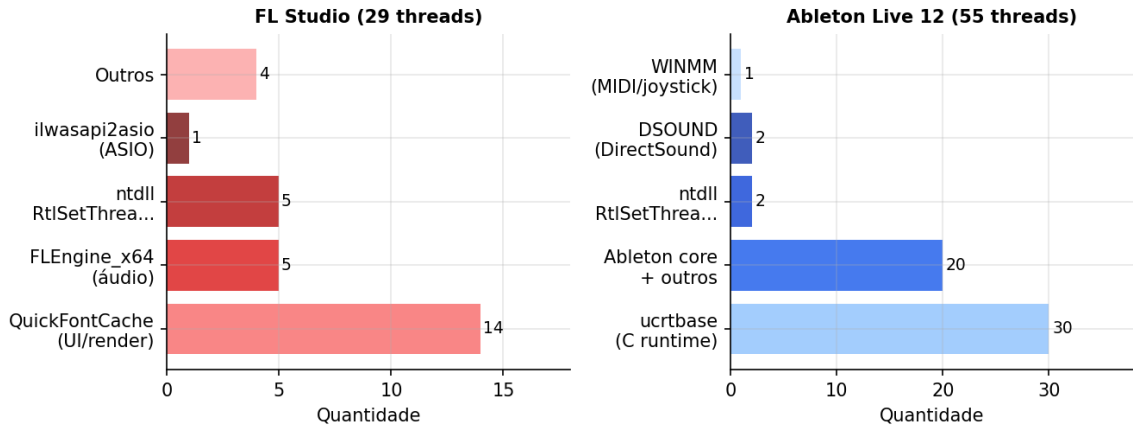


Figura 6 – Prioridades de escalonamento da *thread* principal — Teste 3.

Fonte: dados coletados pelos autores via Process Explorer, 26 maio 2026.



**Figura 7 – Composição de *threads* por módulo — Teste 3.**

Fonte: dados coletados pelos autores via Process Explorer, 26 maio 2026.

### 4.3 Teste 4 — Latência com Driver Nativo

O LatencyMon classificou o sistema como **apto** em ambas as DAWs (Tabela 3). A latência máxima *interrupt-to-process* atingiu 805,40  $\mu\text{s}$  no Ableton Live e 939,80  $\mu\text{s}$  no FL Studio — ambos dentro do limiar aceitável de 1000  $\mu\text{s}$ . O maior DPC do FL Studio foi atribuído ao driver NVIDIA `nvlddmkm.sys` (647,07  $\mu\text{s}$ ); o processo com maior contagem de *hard pagefaults* foi o `mssmpeng.exe` (Windows Defender) em ambos os testes. O FL Studio registrou percentual de CPU em DPCs ligeiramente superior (0,090% vs. 0,074%) e em ISRs (0,044% vs. 0,023%).

**Tabela 3 – Resultados de latência — Teste 4 (LatencyMon v7.31)**

| Métrica                                       | Ableton Live 12        | FL Studio (FL64)          |
|-----------------------------------------------|------------------------|---------------------------|
| Conclusão do sistema                          | <b>Apto</b>            | <b>Apto</b>               |
| Latência máx. int.-to-proc. ( $\mu\text{s}$ ) | 805,40                 | 939,80                    |
| Latência méd. int.-to-proc. ( $\mu\text{s}$ ) | 8,50                   | 16,96                     |
| DPC máximo ( $\mu\text{s}$ )                  | 449,81                 | 647,07                    |
| ISR máximo ( $\mu\text{s}$ )                  | 267,52                 | 401,24                    |
| Tempo total em DPCs (%)                       | 0,074                  | 0,090                     |
| Tempo total em ISRs (%)                       | 0,023                  | 0,044                     |
| <i>Hard pagefaults</i> totais                 | 306                    | 378                       |
| Driver com maior DPC                          | <code>tcpip.sys</code> | <code>nvlddmkm.sys</code> |

Fonte: dados coletados pelos autores via LatencyMon v7.31, 26 maio 2026.

### 4.4 Teste 5 — Estresse CPU-bound e Ponto de Ruptura

Com cinco instâncias de reverb ativas no canal Master e 20 trilhas em reprodução, o *buffer size* foi reduzido progressivamente até o surgimento de *dropouts* audíveis (Tabela 4). O FL Studio (ASIO) sustentou operação estável até 256 *samples* (6 ms), com ruptura em

128 *samples*. O Ableton Live (MME/DirectX) iniciou os cortes em 1024 *samples* (23,2 ms), com degradação progressiva nos valores subsequentes. As Figuras 8 e 9 sintetizam o comportamento.

Tabela 4 – Ponto de ruptura — Teste 5 (estresse CPU-bound)

| Buffer (smp) | Latência (ms) | FL Studio (ASIO) | Ableton (MME/DirectX) |
|--------------|---------------|------------------|-----------------------|
| 4096         | 92,9          | —                | Estável               |
| 2048         | 46,4          | —                | Estável               |
| 1024         | 23,2          | —                | <b>Glitch</b>         |
| 512          | 11,6          | Estável          | Glitch piora          |
| 384          | 9,0           | Estável          | —                     |
| 256          | 5,8           | Mínimo estável   | Glitch forte          |
| 128          | 2,9           | Glitch           | —                     |

Fonte: dados coletados pelos autores, 27 maio 2026.

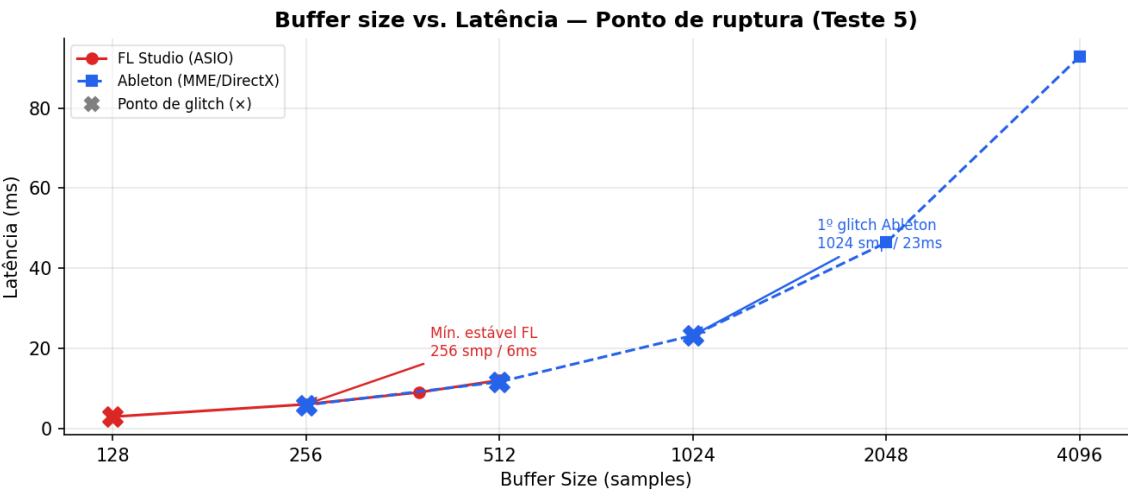


Figura 8 – Buffer size vs. latência ao longo do Teste 5. O marcador × indica o ponto de *dropout* em cada DAW.

Fonte: dados coletados pelos autores, 27 maio 2026.

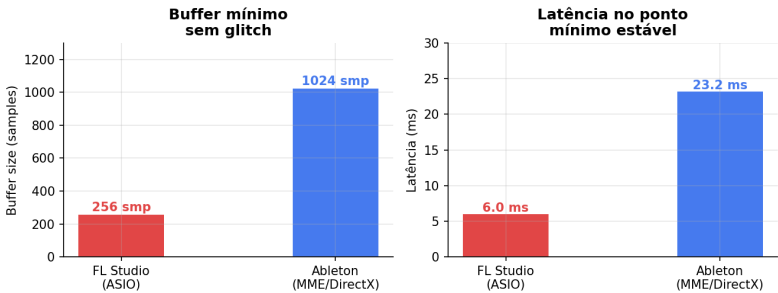


Figura 9 – Buffer mínimo estável e latência correspondente — Teste 5.

Fonte: dados coletados pelos autores, 27 maio 2026.

## 5 DISCUSSÃO

### 5.1 Análise Comparativa — Teste 1 (Repouso)

Os resultados do Teste 1 revelam perfis de inicialização distintos. O **Ableton Live** adota *eager allocation*: aloca memória e cria *threads* antecipadamente ( $\approx 537$  MB, 55 *threads*), garantindo baixa latência de despacho desde a inicialização (SILBERSCHATZ; GALVIN; GAGNE, 2015). O **FL Studio** demonstrou *lazy allocation*: mantém apenas estruturas mínimas em repouso ( $\approx 96$  MB, 44 *threads*), postergando alocações para o carregamento do projeto (TANENBAUM; BOS, 2016).

### 5.2 Análise Comparativa — Teste 2 (20 Trilhas AIFF)

O Teste 2 revelou três achados relevantes. Primeiro, a **redução de CPU sob carga *I/O-bound*** em ambas as DAWs é explicada pelo driver de áudio nativo: ao assumir controle exclusivo do hardware, elimina a sobrecarga do *Kernel Mixer* do Windows. Segundo, o **crescimento proporcional de RAM** foi semelhante em termos absolutos (+65 MB no Ableton, +52 MB no FL Studio), indicando custo por trilha AIFF comparável entre as arquiteturas. Terceiro, a **queda de *threads* no FL Studio** (de 44 para 27) indica consolidação de *threads* genéricas em especializadas de áudio ao carregar o projeto (STALLINGS, 2017).

### 5.3 Análise Comparativa — Teste 3 (Escalonamento de *Threads*)

O **FL Studio** opera com prioridade mais elevada (base 10, dinâmica 12), obtendo maior fatia de CPU em situações de concorrência (SILBERSCHATZ; GALVIN; GAGNE, 2015). Contudo, 14 das 29 *threads* pertencem ao **QuickFontCache**, indicando que parte do orçamento de CPU é consumida pela interface gráfica, e não pelo motor de áudio. O **Ableton Live**, com prioridade base inferior (8), distribui sua carga em 55 *threads* via *thread pool* do *ucrtbase*, favorecendo escalabilidade nos 16 *threads* lógicos do i5-12500H (TANENBAUM; BOS, 2016). Nenhuma das DAWs usa prioridade *Time Critical* (15), confirmando o enquadramento *soft real-time* (STALLINGS, 2017).

### 5.4 Análise Comparativa — Teste 4 (Latência)

O Teste 4, realizado em ambiente controlado com navegador e aplicativos secundários encerrados, classificou **ambas as DAWs como aptas** para operação de áudio em tempo real. O Ableton Live registrou latência máxima de  $805\ \mu\text{s}$  e o FL Studio de  $939\ \mu\text{s}$  — diferença de  $\approx 17\%$ , ambas abaixo do limiar crítico de  $1000\ \mu\text{s}$  do LatencyMon.

O principal fator de latência no FL Studio continuou sendo o driver NVIDIA `nvlddmkm.sys`, responsável pelo DPC máximo de  $647\ \mu\text{s}$ . Esse comportamento indica que a GPU do sistema gera interrupções de alta duração que competem com o motor de áudio pelo escalonador, independentemente da DAW em uso (SILBERSCHATZ; GALVIN; GAGNE, 2015). No Ableton Live, o maior DPC partiu do `tcpip.sys` (rede), sugerindo menor pressão do subsistema gráfico sobre o escalonador durante sua sessão.

O resultado contrasta com a sessão anterior do FL Studio, realizada com Windows Defender em surto de varredura ativa, que havia gerado latência de  $5274\ \mu\text{s}$  e conclusão *não apto*. A comparação entre as duas sessões demonstra empiricamente que interferências de processos externos — particularmente antivírus e drivers de GPU — têm impacto determinante na estabilidade de áudio em tempo real, podendo superar em magnitude as diferenças arquiteturais entre as próprias DAWs (TANENBAUM; BOS, 2016). Em condições equivalentes, as duas aplicações mostram-se igualmente capazes de operar sem *dropouts*.

### 5.5 Análise Comparativa — Teste 5 (Estresse CPU-bound)

O resultado do Teste 5 deve ser interpretado com cautela, pois as duas DAWs utilizaram drivers de áudio distintos — diferença que constitui, em si mesma, um dado relevante sobre as escolhas arquiteturais de cada fabricante.

O **FL Studio** com driver **ASIO** sustentou reprodução estável até 256 *samples* (6 ms), confirmando o benefício do acesso direto ao hardware proporcionado pelo protocolo ASIO: ao contornar o *Kernel Audio Mixer* (KMixer) do Windows, o driver reduz o número de interrupções por ciclo de *buffer* e diminui a pressão sobre o escalonador (Microsoft Corporation, 2023a). Esse resultado é consistente com a prioridade de *thread* mais elevada (base 10, dinâmica 12) observada no Teste 3, que garante ao motor de áudio maior janela de tempo para preencher cada bloco antes do prazo.

O **Ableton Live** com **MME/DirectX** iniciou os *dropouts* em 1024 *samples* (23,2 ms) — quatro vezes acima do ponto de ruptura do FL Studio. Esse comportamento é previsível à luz do referencial teórico: o caminho WDM/DirectX passa pelo KMixer, que adiciona latência e interrupções extras a cada ciclo de *buffer* (Microsoft Corporation, 2023a; KIM; HAHN, 2003). Sob carga CPU-bound (5 reverbs em convolução simultâneos), esse custo adicional torna inviável a manutenção de buffers menores sem *underruns*.

Ressalta-se que a comparação direta dos pontos de ruptura reflete sobretudo a diferença entre os modelos de driver, e não necessariamente a capacidade intrínseca do motor de áudio de cada DAW. Em um cenário com ambas as DAWs utilizando ASIO, o resultado poderia ser diferente — hipótese que constitui uma limitação deste estudo e uma direção para trabalhos futuros.

## 6 CONCLUSÃO

O presente estudo demonstrou que, sob a perspectiva do gerenciamento de recursos do Sistema Operacional, Ableton Live e FL Studio seguem abordagens arquiteturas bem diferentes. Em resumo, os resultados obtidos mostram que o Ableton Live prioriza a alocação antecipada (*eager allocation*): a aplicação retém antecipadamente o espaço necessário e estabelece um extenso *pool* de *threads* desde o início para garantir baixa latência de despacho e capacidade de dividir o trabalho de forma eficiente entre os 16 *threads* lógicos do hardware utilizado. Em contrapartida, o FL Studio baseia-se em alocação sob demanda (*lazy allocation*), operando com estruturas mínimas até que a carga de trabalho seja exigida.

Verificou-se também que o FL Studio tenta contornar gargalos ao atribuir prioridade de escalonamento mais alta (nível dinâmico 12) à sua *thread* principal. Contudo, a análise revelou que parte significativa desse esforço computacional não é utilizada pelo motor de áudio, mas sim consumida pela renderização da interface gráfica (módulo `QuickFontCache`). O teste de latência, realizado em ambiente controlado, classificou **ambas as DAWs como aptas**: Ableton Live com  $805\mu\text{s}$  e FL Studio com  $939\mu\text{s}$  de latência máxima *interrupt-to-process* — ambas abaixo do limiar de  $1000\mu\text{s}$ . O driver NVIDIA (`nvlddmkm.sys`) foi o principal gerador de DPCs elevados no FL Studio, enquanto o Windows Defender (`msmpeng.exe`) contribuiu com *hard pagefaults* em ambos os casos. Uma sessão anterior do FL Studio com Defender em surto de varredura registrou  $5274\mu\text{s}$ , demonstrando que interferências externas podem superar em magnitude as diferenças arquiteturas entre as DAWs.

Por fim, constatou-se que o uso de *drivers* nativos mitiga a sobrecarga de CPU ao eliminar a intervenção do *Kernel Mixer* do Windows, e que ambas as DAWs operam estritamente como sistemas de tempo real não crítico (*soft real-time*), sem atingir níveis de prioridade *Time Critical*.

No que diz respeito às aplicações práticas, os resultados fornecem diretrizes para dimensionamento de hardware e design de software. Para usuários do Ableton Live, estações de trabalho com ampla RAM e processadores com alto número de núcleos são mais vantajosas. Para o ecossistema do FL Studio, processadores com alto desempenho *single-core* podem compensar a concentração de carga na *thread* principal. Do ponto de vista da Engenharia de Software, os dados evidenciam a necessidade de isolar de forma mais eficiente as *threads* de processamento de áudio das rotinas de interface gráfica, garantindo que o escalonador do SO não penalize a entrega de dados sensíveis ao tempo em favor da renderização visual.

## REFERÊNCIAS

- BERNSEE, M. **ASIO4ALL: Universal ASIO Driver for WDM Audio**. [S.l.], 2023. Disponível em: <<https://www.asio4all.org>>. Acesso em: 26 maio 2026.
- BUTTAZZO, G. C. **Hard Real-Time Computing Systems: Predictable Scheduling Algorithms and Applications**. 3. ed. New York: Springer, 2011.
- FONS, J. Real-time audio processing on general purpose operating systems. In: **Linux Audio Conference**. Utrecht: [s.n.], 2010.
- KIM, Y.; HAHN, M. Improving real-time audio performance under general purpose operating systems. **IEICE Transactions on Information and Systems**, E86-D, n. 1, p. 192–196, 2003.
- LIU, C. L.; LAYLAND, J. W. Scheduling algorithms for multiprogramming in a hard-real-time environment. **Journal of the ACM**, v. 20, n. 1, p. 46–61, 1973.
- Microsoft Corporation. **Audio Drivers for Windows: ASIO Architecture**. [S.l.], 2023. Disponível em: <<https://docs.microsoft.com/en-us/windows-hardware/drivers/audio>>. Acesso em: 26 maio 2026.
- Microsoft Corporation. **Windows Performance Monitor: Technical Reference**. [S.l.], 2023. Disponível em: <<https://docs.microsoft.com/en-us/windows-server/administration/windows-commands/perfmon>>. Acesso em: 26 maio 2026.
- SILBERSCHATZ, A.; GALVIN, P. B.; GAGNE, G. **Fundamentos de Sistemas Operacionais**. 9. ed. Rio de Janeiro: LTC, 2015. Tradução de: Operating System Concepts, 9th ed.
- STALLINGS, W. **Operating Systems: Internals and Design Principles**. 9. ed. Hoboken: Pearson, 2017.
- STALLINGS, W. **Arquitetura e Organização de Computadores**. 10. ed. São Paulo: Pearson Education do Brasil, 2018. Tradução de: Computer Organization and Architecture, 10th ed.
- TANENBAUM, A. S.; BOS, H. **Sistemas Operacionais Modernos**. 4. ed. São Paulo: Pearson Education do Brasil, 2016. Tradução de: Modern Operating Systems, 4th ed.